
LAPORAN PRAKTIKUM TEKNOLOGI IOT II



Disusun Oleh:

Nama: RAHMAT FADILLAH PUTRA

Kelas: Teknologi Rekayasa Internet (3C)

NPM: 25782079

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG 2026

1.4.1 Kebutuhan Perangkat Keras

1. Board mikrokontroler ESP32 / NodeMCU ESP8266 (dalam praktikum ini disimulasikan via Wokwi).
2. 1 buah LED 5mm.
3. 1 buah Resistor 220 Ω (pembatas arus LED).
4. 1 buah Resistor 10 k Ω (pull-down push button).
5. 1 buah Push Button / Tactile Switch.
6. Kabel jumper secukupnya.
7. Aplikasi Wokwi Simulator (berbasis web) sebagai lingkungan simulasi rangkaian dan kode program.

1.5 Praktikum

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

1. Rangkaian

- Kaki Anoda (kaki panjang) LED dihubungkan ke pin D5 (GPIO5) mikrokontroler.
- Kaki Katoda (kaki pendek) LED dihubungkan ke salah satu ujung Resistor 220 Ω .
- Ujung Resistor lainnya dihubungkan ke jalur GND.

2. Kode Program

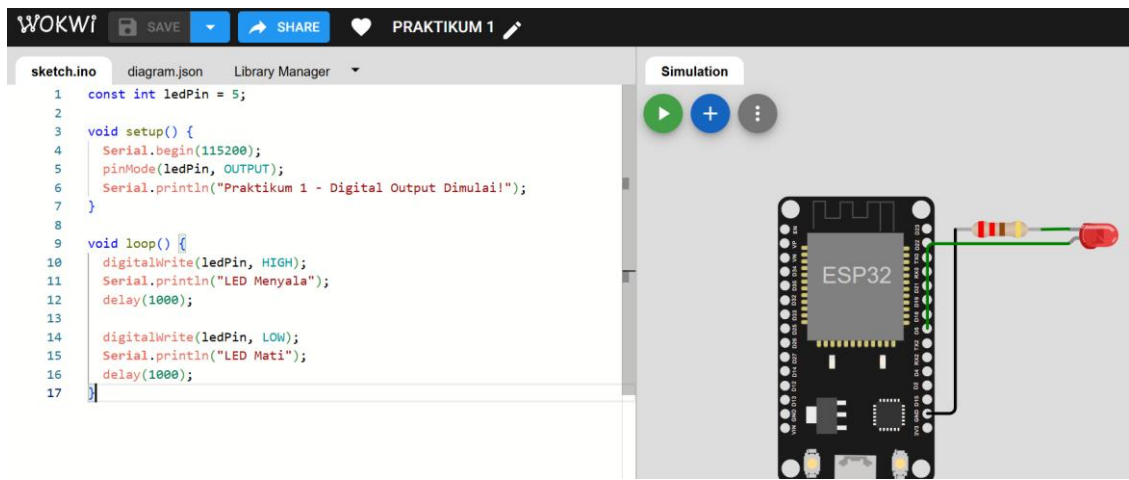
```
const int ledPin = 5;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!");
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala");
  delay(1000);

  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati");
  delay(1000);
}
```

3. Hasil Simulasi



Gambar 1. Simulasi rangkaian Praktikum 1 pada Wokwi — LED terhubung ke pin D5 (GPIO5) melalui resistor 220 Ω

4. Pembahasan

Berdasarkan hasil simulasi pada Wokwi, program berhasil dijalankan dengan baik. Ketika fungsi `digitalWrite(ledPin, HIGH)` dieksekusi, LED menyala karena mendapatkan tegangan penuh dari pin D5. Setelah `delay(1000)` atau jeda 1 detik, fungsi `digitalWrite(ledPin, LOW)` dieksekusi sehingga LED padam kembali. Proses ini berulang terus-menerus di dalam blok `loop()`, menghasilkan efek LED berkedip (blinking) dengan interval 1 detik menyala dan 1 detik mati. Status ini juga tercetak secara sinkron pada Serial Monitor dengan pesan "LED Menyala" dan "LED Mati", yang membuktikan bahwa logika program dan kondisi fisik LED telah sesuai.

1.5.2 Praktikum 2: Digital Input (Membaca Push Button)

1. Rangkaian

Ringkasan koneksi pin pada Praktikum 2 (rangkaian LED dari Praktikum 1 tetap terpasang pada pin D5):

KOMPONEN	PIN MIKROKONTROLER	KETERANGAN
LED (Anoda)	D5 (GPIO5)	Menyalakan LED sebagai Digital Output
LED (Katoda)	GND (via Resistor 220Ω)	Pembatas arus LED
Push Button (kaki 1)	3V3	Sumber tegangan HIGH saat ditekan
Push Button (kaki 2)	D4 (GPIO4)	Dibaca sebagai Digital Input
Push Button (kaki 2) — Pull-down	GND (via Resistor 10kΩ)	Memastikan pin LOW saat tombol tidak ditekan

2. Kode Program

```
const int buttonPin = 4;
const int ledPin = 5;

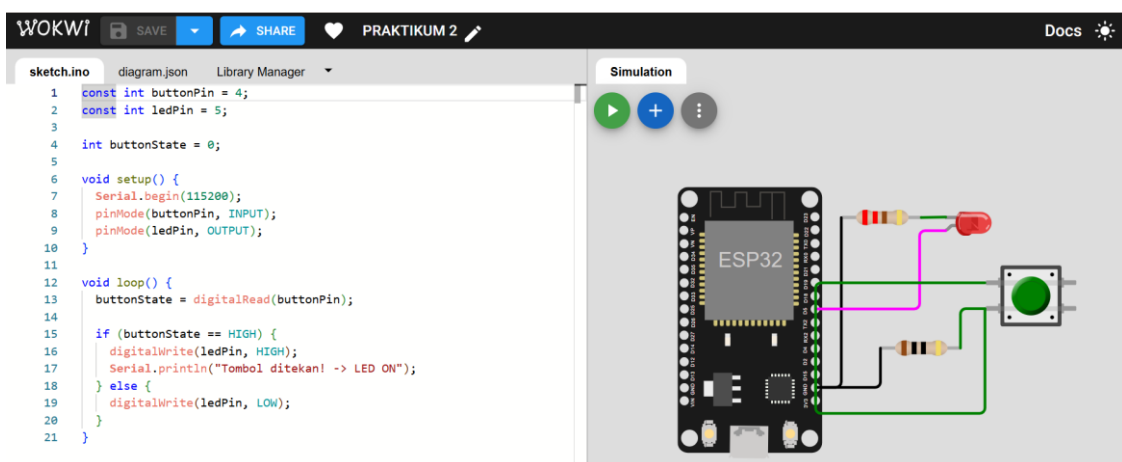
int buttonState = 0;

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Tombol ditekan! -> LED ON");
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

3. Hasil Simulasi



Gambar 2. Simulasi rangkaian Praktikum 2 pada Wokwi — Push button terhubung ke pin D4 (GPIO4) dengan resistor pull-down 10 k Ω

4. Pembahasan

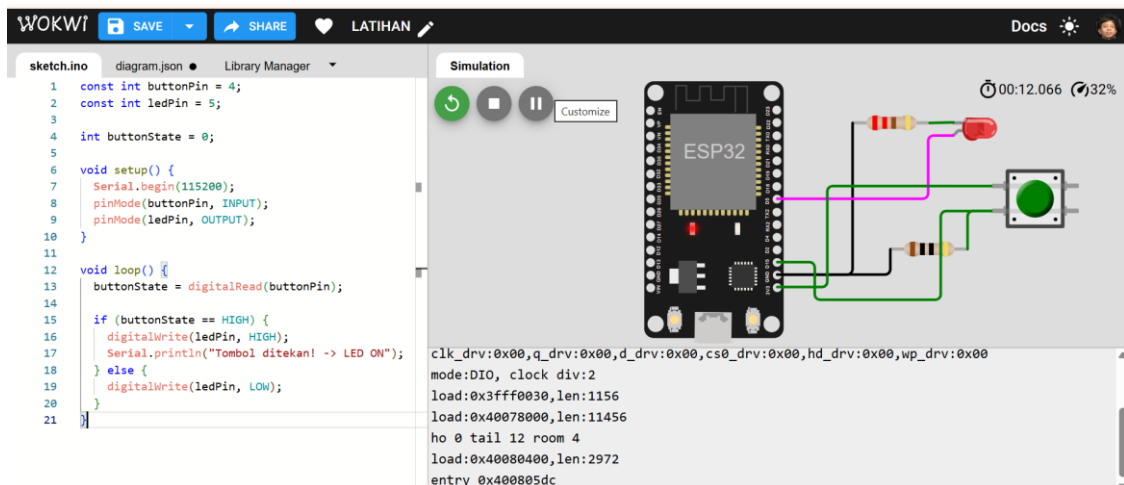
Pada kondisi awal (tombol tidak ditekan), pin D4 ditarik ke GND melalui resistor pull-down 10 k Ω sehingga `digitalRead(buttonPin)` menghasilkan nilai LOW, dan LED dalam keadaan mati (`digitalWrite(ledPin, LOW)`). Ketika tombol ditekan, tegangan 3.3V dari pin 3V3 mengalir langsung menuju pin D4 karena hambatannya jauh lebih kecil dibandingkan resistor pull-down, sehingga pin terbaca HIGH. Kondisi `if (buttonState == HIGH)` kemudian terpenuhi, menyebabkan LED menyala dan pesan "Tombol ditekan! -> LED ON" tercetak pada Serial Monitor. Saat tombol dilepas, pin kembali dipastikan LOW oleh resistor pull-down sehingga LED langsung padam. Perilaku ini konsisten dengan konsep logika Active-High yang dijelaskan pada dasar teori.

1.6 Latihan

Latihan 1 — Pemahaman Logika Pin

Soal: Berdasarkan tabel referensi pinout, jelaskan apa yang mungkin terjadi jika rangkaian Praktikum 2 (Active-High / pull-down) disambungkan ke pin D8 (GPIO15), lalu tombol ditekan tepat pada saat NodeMCU baru diberi daya.

Jawaban: Berdasarkan tabel pinout, pin D8 (GPIO15) memiliki karakteristik khusus, yaitu secara internal ditarik ke GND (pulled to GND) dan proses booting akan gagal apabila pin ini ditarik ke kondisi HIGH saat board baru dinyalakan. Pada rangkaian pull-down Praktikum 2, saat tombol ditekan maka tegangan 3.3V akan mengalir ke pin dan membuatnya bernilai HIGH. Jika hal ini terjadi tepat pada saat proses booting berlangsung, maka akan terjadi konflik logika: ESP menginterpretasikan pin GPIO15 yang bernilai HIGH sebagai perintah untuk masuk ke mode boot yang berbeda (bukan mode boot normal dari flash memory). Akibatnya, board dapat mengalami kegagalan booting (boot loop), tidak menjalankan program dengan normal, atau bahkan tidak merespons sama sekali sampai tombol dilepas dan board di-reset ulang. Oleh karena itu, pin D8 tidak direkomendasikan untuk rangkaian input eksternal seperti push button.



Gambar 3. Rangkaian dasar Praktikum 2 (Active-High/pull-down) pada pin D4 yang dijadikan acuan pembahasan Latihan 1

Latihan 2 — Implementasi Kode (Blink Asimetris)

Soal: Modifikasi kode Praktikum 1 agar LED berkedip tidak simetris — menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms.

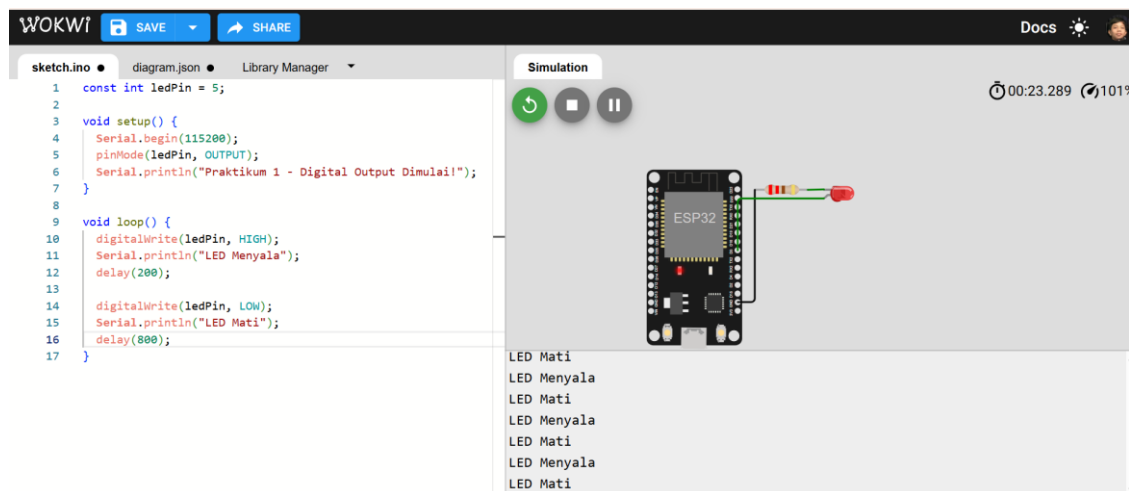
Jawaban: Dua baris fungsi delay() yang diubah adalah delay(1000) setelah LED menyala menjadi delay(200), dan delay(1000) setelah LED mati menjadi delay(800), sebagaimana kode berikut:

```
const int ledPin = 5;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!");
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala");
  delay(200); // LED menyala cepat (200 ms)

  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati");
  delay(800); // LED mati agak lama (800 ms)
}
```



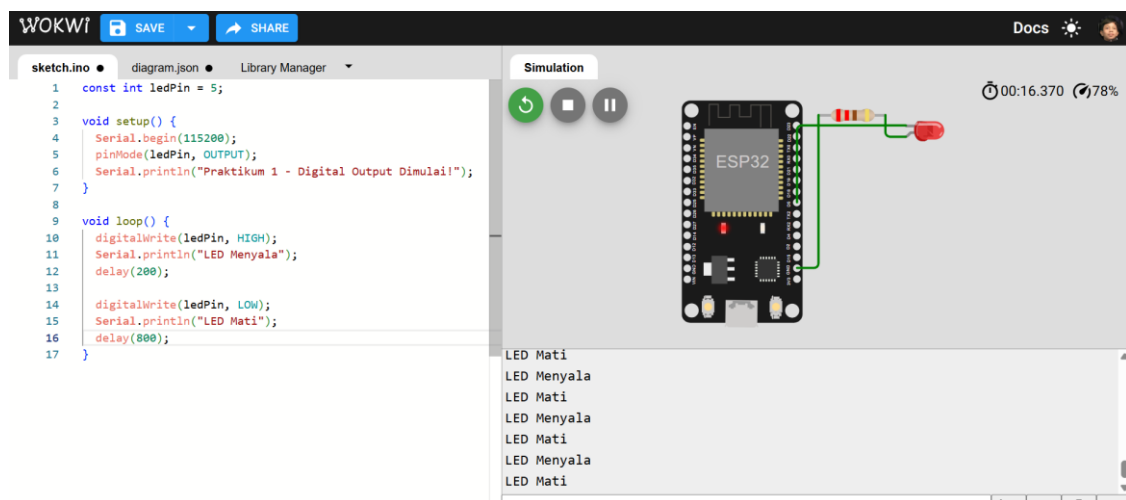
Gambar 4. Hasil pengujian Latihan 2 — LED berkedip asimetris (menyala 200 ms, mati 800 ms)

Berdasarkan hasil simulasi, LED berkedip lebih cepat saat menyala dan bertahan lebih lama saat mati, sesuai dengan perubahan durasi delay() yang dilakukan. Pesan pada Serial Monitor "LED Menyala" dan "LED Mati" juga tercetak silih berganti mengikuti interval waktu yang baru.

Latihan 3 — Analisis Sirkuit (LED Terbalik)

Soal: Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 kaki LED dipasang terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami error kompilasi? Mengapa LED tidak menyala?

Jawaban: Program tidak akan mengalami error kompilasi, sebab compiler Arduino IDE hanya memeriksa kebenaran sintaks dan logika kode program, bukan kebenaran arah pemasangan komponen fisik pada rangkaian. Namun secara fisik, LED tidak akan menyala. Hal ini disebabkan karena LED merupakan komponen dioda yang hanya dapat mengalirkan arus listrik searah (forward bias), yaitu dari Anoda menuju Katoda. Apabila posisi kaki dibalik (Anoda ke GND dan Katoda ke pin D1), maka pada saat pin diberi logika HIGH, arus justru berusaha mengalir dari Katoda menuju Anoda (reverse bias). Pada kondisi reverse bias dengan tegangan sekecil 3.3V, dioda LED akan menahan aliran arus (dianggap seperti saklar terbuka) sehingga tidak ada arus yang mengalir dan LED tidak menyala, meskipun program tetap berjalan normal dan tidak menampilkan pesan error apa pun.



Gambar 5. Rangkaian acuan Praktikum 1 (pemasangan normal) yang digunakan sebagai pembanding pada analisis Latihan 3

Latihan 4 — Pengembangan Algoritma (Logika Terbalik)

Soal: Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: saat tombol TIDAK ditekan, LED menyala; saat tombol DITEKAN, LED justru mati.

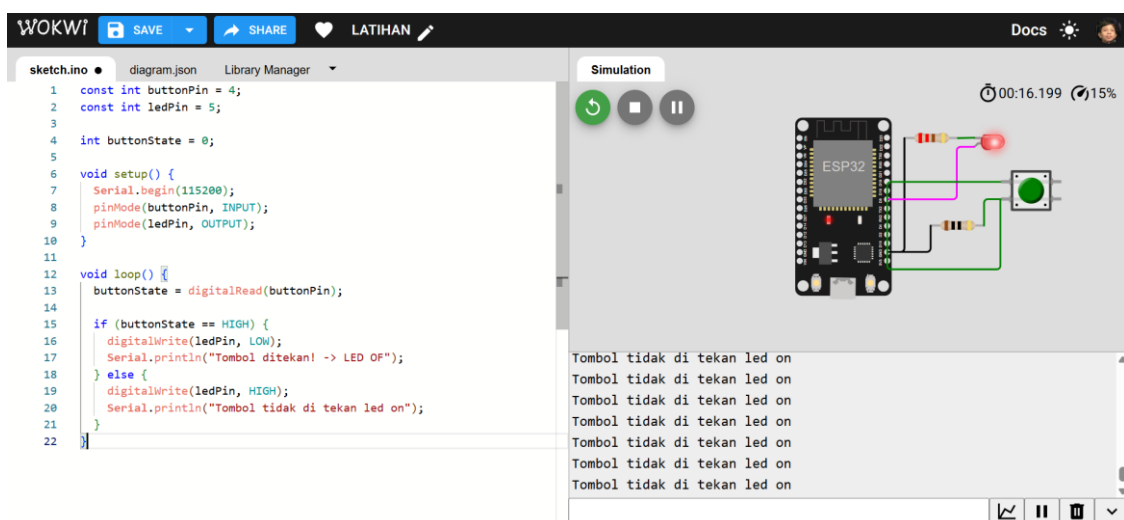
```
const int buttonPin = 4;
const int ledPin = 5;

int buttonState = 0;

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, LOW);
    Serial.println("Tombol ditekan! -> LED OFF");
  } else {
    digitalWrite(ledPin, HIGH);
    Serial.println("Tombol tidak ditekan -> LED ON");
  }
}
```



Gambar 6. Hasil pengujian Latihan 4 — LED menyala saat tombol tidak ditekan dan mati saat tombol ditekan

Berdasarkan hasil simulasi, ketika tombol tidak ditekan (buttonState bernilai LOW), program masuk ke blok else sehingga LED diberi logika HIGH dan menyala, disertai pesan "Tombol tidak ditekan -> LED ON" pada Serial Monitor. Sebaliknya, ketika tombol ditekan (buttonState bernilai HIGH), blok if terpenuhi sehingga LED diberi logika LOW dan padam, disertai pesan "Tombol ditekan! -> LED OFF". Perilaku ini membuktikan bahwa logika if-else berhasil dibalik sepenuhnya dari kondisi awal pada Praktikum 2.

1.7 Tugas — Modifikasi Sistem Sakelar Toggle (Latching)

Tujuan: Mengimplementasikan logika pelacakan status (state tracking) sehingga tombol berfungsi layaknya sakelar lampu ruangan (toggle/latching), berbeda dengan perilaku momentary pada Praktikum 2, dengan konfigurasi hardware yang sama persis (rangkaian pull-down).

Kode Program

```
const int ledPin = 5;      // LED terhubung ke GPIO 23
const int buttonPin = 4;   // Push button terhubung ke GPIO 22

bool ledState = false;

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);

  // Kondisi awal LED mati
  digitalWrite(ledPin, LOW);

  Serial.println("=== SISTEM SAKELAR TOGGLE ESP32 ===");
  Serial.println("LED awal: OFF");
}

void loop() {
  int buttonState = digitalRead(buttonPin);

  // Jika tombol ditekan
  if (buttonState == HIGH) {

    // Mengubah kondisi LED (toggle status tersimpan di ledState)
    ledState = !ledState;
```

```

digitalWrite(ledPin, ledState);

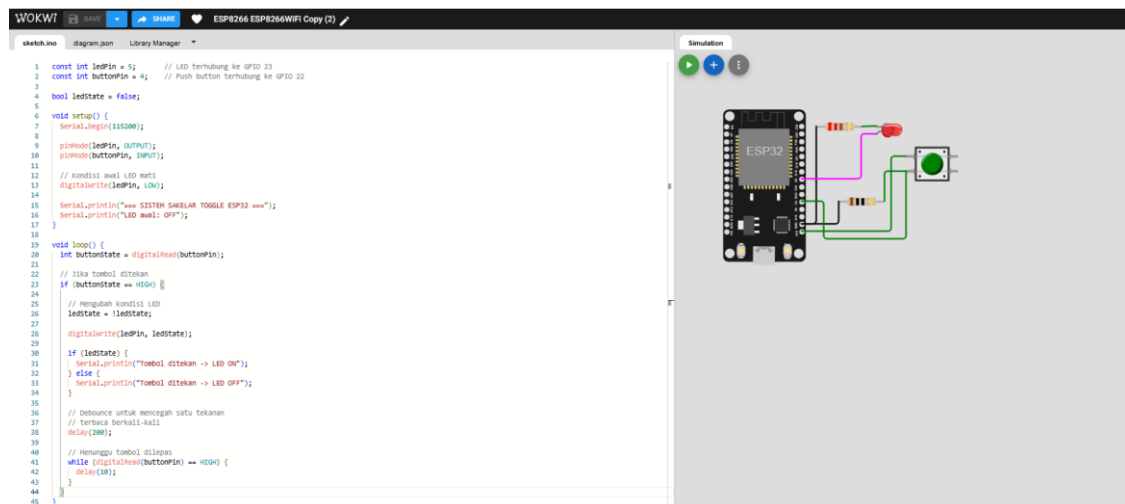
if (ledState) {
    Serial.println("Tombol ditekan -> LED ON");
} else {
    Serial.println("Tombol ditekan -> LED OFF");
}

// Debounce untuk mencegah satu tekanan
// terbaca berkali-kali
delay(200);

// Menunggu tombol dilepas, agar toggle
// hanya terjadi sekali per penekanan
while (digitalRead(buttonPin) == HIGH) {
    delay(10);
}
}
}

```

Hasil Simulasi



Gambar 7. Simulasi Tugas — sistem sakelar toggle (latching) pada Wokwi

Pembahasan

Program menggunakan variabel boolean `ledState` untuk menyimpan (melacak) status LED saat ini, apakah sedang menyala (`true`) atau mati (`false`). Setiap kali tombol terdeteksi ditekan (`buttonState == HIGH`), nilai `ledState` dibalik menggunakan operator negasi (`ledState = !ledState`), kemudian nilai tersebut langsung diterapkan ke LED melalui `digitalWrite(ledPin, ledState)`. Dengan mekanisme ini, satu kali tekan-lepas tombol akan mengubah status LED secara permanen (latching), berbeda dengan Praktikum 2 yang bersifat momentary (LED hanya menyala selama tombol ditahan).

Untuk mengatasi debouncing atau pantulan mekanis pada tombol fisik, ditambahkan `delay(200)` tepat setelah perubahan status terdeteksi. Selanjutnya, instruksi `while (digitalRead(buttonPin) == HIGH) { delay(10); }` digunakan untuk menahan program agar menunggu hingga tombol benar-benar dilepas sebelum kembali membaca status tombol pada iterasi `loop()` berikutnya. Tanpa baris ini, satu kali tekanan yang ditahan agak lama berpotensi terbaca berkali-kali dan menyebabkan LED berkedip-kedip (flickering) alih-alih berpindah status secara bersih. Dengan kombinasi debounce dan penantian pelepasan tombol ini, perubahan status ON/OFF hanya terjadi tepat satu kali untuk setiap satu kali tekanan singkat, sesuai dengan kriteria keberhasilan tugas.